SIMATS ENGINEERING

Engineer to Excel

SIMATS

Saveetha Institute of Medical And Technical Sciences

(Declared as Deemed to be University under Section 3 of UGC Act 1956)

CO2-ASSIGNMENT-2

NAME: P. DEEKSHANA

REG NO: 192425104

COURSE CODE: CSA0601

COURSE NAME: DESIGN AND ANALYSIS OF ALGORITHMS

➢ Q1 – Sorting Under Different Data Conditions

1. Aim

To implement Selection Sort, Bubble Sort, and Insertion Sort using Python and compare their performance under different input conditions by counting comparisons and movements.

2. Introduction

Sorting is the process of arranging data in ascending or descending order. Sorting algorithms are used in databases, searching applications, and data processing.

This experiment compares three algorithms using sorted, reverse-sorted, and nearly sorted datasets. Their performance is evaluated based on the number of comparisons, movements, and time complexity.

3. Algorithm Description

- Selection Sort: Finds the smallest element in the unsorted portion and swaps it into the correct position.
- Bubble Sort: Compares adjacent elements and swaps them when they are in the wrong order. It stops early if no swaps are needed.
- Insertion Sort: Takes each element and inserts it into the correct position within the already sorted portion.

5. Python Implementation

The following program implements all three algorithms and counts their comparisons and movements. The input list is copied before each execution so that every algorithm receives the same original data.

```python
def selection_sort(arr):
    comparisons = 0
    movements = 0
    n = len(arr)

    for i in range(n - 1):
        min_index = i

        for j in range(i + 1, n):
            comparisons += 1

            if arr[j] < arr[min_index]:
                min_index = j

        if min_index != i:
            arr[i], arr[min_index] = arr[min_index], arr[i]
            movements += 1

    return arr, comparisons, movements


def bubble_sort(arr):
    comparisons = 0
    movements = 0
    n = len(arr)

    for i in range(n - 1):
        swapped = False

        for j in range(n - i - 1):
            comparisons += 1

            if arr[j] > arr[j + 1]:
                arr[j], arr[j + 1] = arr[j + 1], arr[j]
                movements += 1
                swapped = True

        if not swapped:
            break

    return arr, comparisons, movements


def insertion_sort(arr):
    comparisons = 0
    movements = 0
    n = len(arr)

    for i in range(1, n):
        key = arr[i]
        j = i - 1

        while j >= 0:
            comparisons += 1

            if arr[j] > key:
                arr[j + 1] = arr[j]
                movements += 1
                j -= 1
            else:
                break

        arr[j + 1] = key
        movements += 1

    return arr, comparisons, movements


datasets = {
    "Sorted": [1, 2, 3, 4, 5, 6],
    "Reverse Sorted": [6, 5, 4, 3, 2, 1],
    "Nearly Sorted": [1, 2, 4, 3, 5, 6]
}

algorithms = {
    "Selection Sort": selection_sort,
    "Bubble Sort": bubble_sort,
    "Insertion Sort": insertion_sort
}

for dataset_name, values in datasets.items():
    print("\nDataset:", dataset_name)

    for name, algorithm in algorithms.items():
        result, comparisons, movements = algorithm(values.copy())

        print(name)
        print("Sorted array:", result)
        print("Comparisons:", comparisons)
        print("Movements:", movements)
```

5. Explanation of the Program

- The three functions implement the respective sorting algorithms.
- comparisons counts comparisons between elements.
- movements counts swaps for Selection and Bubble Sort, and shifts plus key placements for Insertion Sort.
- The datasets dictionary stores three different input arrangements.
- The algorithms dictionary stores the algorithm functions.
- values.copy() ensures that each algorithm receives the original dataset.
- The program displays the sorted array and operation counts for every test.

6. Sample Output

The following shows the expected output for the already sorted dataset.

Dataset: Sorted

Selection Sort

Sorted array: [1, 2, 3, 4, 5, 6]

Comparisons: 15

Movements: 0

Bubble Sort

Sorted array: [1, 2, 3, 4, 5, 6]

Comparisons: 5

Movements: 0

Insertion Sort

Sorted array: [1, 2, 3, 4, 5, 6]

Comparisons: 5

Movements: 5

The program also displays the results for reverse-sorted and nearly sorted datasets.

Output



7. Experimental Results

The following are expected operation counts for the program.

Dataset	Algorithm	Comparisons	Movements
Sorted	Selection Sort	15	0
Sorted	Bubble Sort	5	0
Sorted	Insertion Sort	5	5
Reverse sorted	Selection Sort	15	3
Reverse sorted	Bubble Sort	15	15
Reverse sorted	Insertion Sort	15	20
Nearly sorted	Selection Sort	15	1
Nearly sorted	Bubble Sort	9	1
Nearly sorted	Insertion Sort	6	6

8. Analysis

- Selection Sort: Performs the same number of comparisons for all three datasets but uses relatively few swaps.
- Bubble Sort: Performs well on sorted data because it stops when no swaps are needed. Reverse-sorted data requires more swaps.
- Insertion Sort: Performs well on sorted and nearly sorted data because fewer shifts are required.

9. Time Complexity

Algorithm	Best Case	Average Case	Worst Case
Selection Sort	O(n^2)	O(n^2)	O(n^2)
Bubble Sort	O(n)	O(n^2)	O(n^2)
Insertion Sort	O(n)	O(n^2)	O(n^2)

Bubble Sort achieves linear best-case time because the program uses early stopping. Insertion Sort also achieves linear best-case time when the input is already sorted.

10. Conclusion

The experiment compares three sorting algorithms under different input conditions. Selection Sort maintains a fixed comparison count, Bubble Sort benefits from early stopping, and Insertion Sort performs efficiently on nearly sorted data.

Therefore, Insertion Sort is a suitable choice for small, nearly sorted datasets. The results show that algorithm performance depends on the arrangement of the input data as well as the number of elements.

➢ Q2. Sequential Search for Aircraft Maintenance Tool Tracking

1. Aim

To implement Sequential Search using Python and evaluate its suitability for an aircraft maintenance tool-tracking system by measuring the number of comparisons for successful and unsuccessful searches.

2. Introduction

Sequential Search, also called Linear Search, checks each element until the target is found or the list ends. In aircraft maintenance, it can locate tool records using Tool IDs. It is simple and does not require sorted data, but searching large lists may take more time.

3. Problem Statement

An aircraft maintenance department maintains a continuously changing list of tool records. The system must locate tool records based on their Tool ID.

The experiment investigates:

- Successful searches where the required Tool ID exists.
- Unsuccessful searches where the Tool ID does not exist.
- Searches for records at the beginning, middle, and end of a list.
- The effect of increasing the number of records.
- Whether Sequential Search remains appropriate when the system expands from one hangar to 20 maintenance facilities.

The number of comparisons is used to evaluate search performance.

4. Python Program

```
def sequential_search(records, target_id):
    comparisons = 0
    for record in records:
        comparisons += 1

        if record["Tool ID"] == target_id:
            return record, comparisons

    return None, comparisons

def create_records(n):
    records = []

    for i in range(1, n + 1):
        record = {
            "Tool ID": "T" + str(i).zfill(4),
            "Technician ID": "TECH" + str((i % 10) + 1),
            "Issue Time": "09:30 AM",
            "Work Bay": "Bay " + str((i % 5) + 1)
        }
        records.append(record)

    return records

sizes = [10, 100, 1000]

for size in sizes:
    records = create_records(size)
    print("\nNumber of records:", size)

    targets = [
        ("First", records[0]["Tool ID"]),
        ("Middle", records[size // 2]["Tool ID"]),
        ("Last", records[-1]["Tool ID"]),
        ("Unsuccessful", "T99999")
    ]
    for position, target in targets:
        result, comparisons = sequential_search(records, target)
        print(position, "Search")
        print("Target:", target)
        print("Comparisons:", comparisons)
        if result is not None:
            print("Record found:", result)
        else:
            print("Record not found")
```

Output

```
Number of records: 10
First Search
Target: T0001
Comparisons: 1
Record found: {'Tool ID': 'T0001', 'Technician ID': 'TECH2', 'Issue Time': '09:30 AM', 'Work Bay': 'Bay 2'}
Middle Search
Target: T0006
Comparisons: 6
Record found: {'Tool ID': 'T0006', 'Technician ID': 'TECH7', 'Issue Time': '09:30 AM', 'Work Bay': 'Bay 2'}
Last Search
Target: T0010
Comparisons: 10
Record found: {'Tool ID': 'T0010', 'Technician ID': 'TECH1', 'Issue Time': '09:30 AM', 'Work Bay': 'Bay 1'}
Unsuccessful Search
Target: T99999
Comparisons: 10
Record not found

Number of records: 100
First Search
Target: T0001
Comparisons: 1
Record found: {'Tool ID': 'T0001', 'Technician ID': 'TECH2', 'Issue Time': '09:30 AM', 'Work Bay': 'Bay 2'}
Middle Search
Target: T0051
Comparisons: 51
Record found: {'Tool ID': 'T0051', 'Technician ID': 'TECH2', 'Issue Time': '09:30 AM', 'Work Bay': 'Bay 2'}
Last Search
Target: T0100
Comparisons: 100
Record found: {'Tool ID': 'T0100', 'Technician ID': 'TECH1', 'Issue Time': '09:30 AM', 'Work Bay': 'Bay 1'}
Unsuccessful Search
Target: T99999
Comparisons: 100
Record not found

Number of records: 1000
First Search
Target: T0001
Comparisons: 1
Record found: {'Tool ID': 'T0001', 'Technician ID': 'TECH2', 'Issue Time': '09:30 AM', 'Work Bay': 'Bay 2'}
Middle Search
Target: T0501
Comparisons: 501
Record found: {'Tool ID': 'T0501', 'Technician ID': 'TECH2', 'Issue Time': '09:30 AM', 'Work Bay': 'Bay 2'}
Last Search
```

5. Explanation of the Program

- create_records() creates sample tool records.
- sequential_search() searches for the required Tool ID.
- The comparisons variable counts the records checked.
- The program tests lists containing 10, 100, and 1,000 records.
- It checks records at the beginning, middle, end, and an absent Tool ID.

6. Experimental Results

The following comparison counts are expected from the program.

Records	First	Middle	Last	Unsuccessful
10	1	6	10	10
100	1	51	100	100
1,000	1	501	1,000	1,000

A successful search can require as few as one comparison when the target is the first record. When the target is the last record, every record must be examined. An unsuccessful search also requires examining the entire list.

7. Analysis

7.1 Effect of Search Position

The position of the target directly affects the number of comparisons. A record at the beginning is located quickly, while a record at the end requires checking all preceding records. If the record is absent, the

algorithm must examine the complete list.

7.2 Effect of Dataset Size

As the list grows from 10 to 1,000 records, the number of comparisons increases for middle, last, and unsuccessful searches. Sequential Search does not require sorting or additional indexing, but repeated searches on large lists can become inefficient.

7.3 Time Complexity

Case	Time complexity	Explanation
Best case	(O(1))	Target is the first record
Average case	(O(n))	Several records must be examined
Worst case	(O(n))	Target is last or absent

The space complexity of the search operation is (O(1)), because it uses only a fixed number of additional variables. The space required to store the records themselves is separate.

8. Suitability for Aircraft Maintenance

For a single hangar with a small number of tool records and relatively infrequent searches, Sequential Search can be an appropriate solution. It is easy to implement, does not require sorted records, and works well when the list changes frequently.

However, expansion to 20 maintenance facilities may result in a much larger number of active tool records and search requests. In that situation, repeatedly examining every record can increase response time. A more suitable design could use a hash table or dictionary indexed by Tool ID, which typically supports near-constant-time lookups. A database index may be more appropriate when records must be shared across facilities and updated by multiple users.

Operating condition	Recommendation
Small list, infrequent searches	Sequential Search
Small, frequently changing list	Sequential Search may be sufficient
Large list, frequent searches	Hash table or indexed database
Multiple maintenance facilities	Centralized database with suitable indexing

These recommendations are based on algorithmic complexity. Actual response times should be measured using representative record counts and search frequencies before making a final deployment decision.

9. Conclusion

The experiment demonstrates that Sequential Search is simple and effective for small datasets, particularly when the target is near the beginning of the list. Its performance decreases as the number of records

increases, because successful searches may require many comparisons and unsuccessful searches require a complete scan.

Therefore, Sequential Search is suitable for a small aircraft maintenance tool-tracking system with limited search activity. For a system expanded across 20 maintenance facilities, a hash table or indexed database should be evaluated to improve lookup efficiency.

➢ Q3. Brute-Force String Matching Algorithm

1. Aim

To implement the Brute-Force String Matching algorithm using Python and evaluate its performance by counting character comparisons for successful and unsuccessful pattern searches.

2. Introduction

String matching finds a pattern within a larger text. It is used in search engines, text editors, and plagiarism detection. The brute-force method checks every possible starting position and compares characters until a match or mismatch occurs.

3. Problem Statement

Given a text and a pattern, determine whether the pattern occurs in the text and identify its starting position. Count the character comparisons performed during the search.

The experiment considers:

- A pattern found at the beginning of the text.
- A pattern found near the end of the text.
- A pattern that does not occur in the text.
- The effect of increasing text length.

4. Algorithm

1. Read the text and the pattern.
2. Calculate their lengths.
3. Place the pattern at the beginning of the text.
4. Compare the pattern characters with the corresponding text characters.
5. If all characters match, record the starting position.
6. If a mismatch occurs, shift the pattern one position to the right.
7. Repeat until all possible positions have been checked.
8. Display the matching positions and the total number of comparisons.

5. Python Program

```python
def brute_force_search(text, pattern):
    n = len(text)
    m = len(pattern)

    comparisons = 0
    positions = []

    if m == 0:
        return positions, comparisons

    for i in range(n - m + 1):
        j = 0

        while j < m:
            comparisons += 1

            if text[i + j] != pattern[j]:
                break

            j += 1

        if j == m:
            positions.append(i)

    return positions, comparisons


text = input("Enter the text: ")
pattern = input("Enter the pattern: ")

positions, comparisons = brute_force_search(text, pattern)

if positions:
    print("Pattern found at positions:", positions)
else:
    print("Pattern not found")

print("Character comparisons:", comparisons)
```

6. Explanation of the Program

- text stores the input string, and pattern stores the target.
- n and m represent their lengths.
- The nested loops compare characters at each position.
- positions stores the matching locations.
- comparisons counts character comparisons.

7. Sample Output

Example 1: Successful search

Enter the text: AABAACAADAABAABA

Enter the pattern: AABA

Pattern found at positions: [0, 9, 12]

Character comparisons: 26

Example 2: Unsuccessful search

Enter the text: COMPUTER

Enter the pattern: DATA

Pattern not found

Character comparisons: 6

Output

```
Python 3.14.7 (tags/v3.14.7:823f032, Aug  5 2026, 10:51:32) [MSC v.1944 64 bit (AMD64)] on win32
Enter "help" below or click "Help" above for more information.
>>>
==================== RESTART: C:/Users/ASUS/Downloads/1.py ====================
Enter the text: AABAACAADAABAABA
Enter the pattern: AABA
Pattern found at positions: [0, 9, 12]
Character comparisons: 30
>>>
==================== RESTART: C:/Users/ASUS/Downloads/1.py ====================
Enter the text: COMPUTER
Enter the pattern: DATA
Pattern not found
Character comparisons: 5
```

8. Experimental Analysis

Text	Pattern	Result	Comparisons
ABCDE	AB	Found at 0	2
ABCDE	DE	Found at 3	6
ABCDE	XY	Not found	4
AAAAA	AAA	Found at 0, 1, 2	9

The number of comparisons depends on the pattern length and the characters matched before a mismatch occurs.

9. Time and Space Complexity

Let (n) be the text length and m be the pattern length.

Case	Time complexity	Explanation
Best case	(O(n))	Each attempted position fails immediately
Average case	Typically (O(n)) for fixed patterns and ordinary data	Most positions require only a few comparisons
Worst case	(O(nm))	Many characters match before each mismatch

The worst case occurs when the text and pattern contain repeated characters that produce long partial matches. The search uses (O(1)) auxiliary space, excluding the list used to store matching positions.

10. Conclusion

The Brute-Force String Matching algorithm provides a straightforward way to find a pattern in a text. It checks every possible starting position and records all occurrences. Its main advantage is simplicity, while its main limitation is the potentially large number of character comparisons.

➢ Q4. Closest Pair of Points Using Brute-Force Algorithm

1. Aim

To implement the Brute-Force Closest Pair of Points algorithm using Python and identify the two points with the minimum Euclidean distance. The performance is evaluated by counting the number of distance calculations.

2. Introduction

The Closest Pair problem identifies the two nearest points in a two-dimensional plane. It is useful in geographical mapping, navigation, and sensor networks. The brute-force method calculates the distance between every possible pair and selects the smallest.

3. Problem Statement

Given a set of two-dimensional points, identify the closest pair, calculate their minimum distance, and count the distance calculations.

4. Euclidean Distance Formula

The distance between two points $P_1 = (x_1, y_1)$ and ($P_2 = (x_2, y_2)$ is calculated using:

$$d(P_1, P_2) = \sqrt{(x_2 - x_1)^2 + (y_2 - y_1)^2}$$

The algorithm calculates this distance for every pair of points and selects the pair with the smallest distance.

5. Algorithm

1. Store all points as coordinate pairs.
2. Initialize the minimum distance to infinity.
3. Select the first point.
4. Compare it with every point that comes after it.
5. Calculate the Euclidean distance for each pair.
6. If the calculated distance is smaller than the current minimum, update the minimum distance and closest pair.
7. Repeat until every unique pair has been examined.
8. Display the closest pair, minimum distance, and number of comparisons.

6. Python Program

```python
import math
def closest_pair(points):
    minimum_distance = float("inf")
    closest_points = None
    comparisons = 0
    n = len(points)
    for i in range(n - 1):
        for j in range(i + 1, n):
            x1 = points[i][0]
            y1 = points[i][1]
            x2 = points[j][0]
            y2 = points[j][1]
            distance = math.sqrt(
                (x2 - x1) ** 2 + (y2 - y1) ** 2
            )
            comparisons += 1
            if distance < minimum_distance:
                minimum_distance = distance
                closest_points = (points[i], points[j])
    return closest_points, minimum_distance, comparisons
points = [
    (2, 3),
    (12, 30),
    (40, 50),
    (5, 1),
    (12, 10),
    (3, 4)
]
pair, distance, comparisons = closest_pair(points)

print("Points:", points)
print("Closest pair:", pair)
print("Minimum distance:", round(distance, 2))
print("Distance calculations:", comparisons)
```

7. Explanation of the Program

- The points list stores the coordinates.
- closest_pair() examines every unique pair.
- minimum_distance stores the smallest distance.
- math.sqrt() calculates Euclidean distance.
- comparisons counts the distance calculations.

8. Sample Output

Points: [(2, 3), (12, 30), (40, 50),

(5, 1), (12, 10), (3, 4)]

Closest pair: ((2, 3), (3, 4))

Minimum distance: 1.41

Distance calculations: 15

The points (2,3) and (3,4) are the closest pair. Their distance is approximately \(1.41\) units.

Output

```
Python 3.14.7 (tags/v3.14.7:823f032, Aug  5 2026, 10:51:32) [MSC v.1944 64 bit (AMD64)] on win32
Enter "help" below or click "Help" above for more information.
>>>
========== RESTART: C:/Users/ASUS/Downloads/1.py ==========
Points: [(2, 3), (12, 30), (40, 50), (5, 1), (12, 10), (3, 4)]
Closest pair: ((2, 3), (3, 4))
Minimum distance: 1.41
Distance calculations: 15
```

9. Experimental Results

For n points, the number of unique pairs is:

$$C = \frac{n(n-1)}{2}$$

The following table shows the expected number of distance calculations for different input sizes.

Number of points	Distance calculations
5	10
10	45
50	1,225
100	4,950
500	124,750
1,000	499,500

The table demonstrates that the number of calculations increases rapidly as the number of points grows. For 1,000 points, the algorithm calculates the distance between 499,500 pairs.

10. Time and Space Complexity

Case	Complexity
Best case	O(n^2)
Average case	O(n^2)
Worst case	O(n^2)
Auxiliary space	O(1)

The algorithm examines every unique pair regardless of the initial arrangement of the points. Therefore, its time complexity is quadratic. The algorithm uses constant auxiliary space when the input points are already stored and only the current closest pair and distance are maintained.

11. Analysis and Discussion

The algorithm is simple and guarantees the closest pair for a set containing at least two points. However, the number of calculations grows quadratically, making it inefficient for large datasets. Divide-and-conquer methods can achieve \(O(n\log n)\) time under standard assumptions.

12. Conclusion

The brute-force method accurately finds the closest pair by examining every unique pair. It is suitable for small datasets, while divide-and-conquer algorithms are preferable for larger inputs.

➢ Q5. Convex Hull Using the Brute-Force Algorithm

1. Aim

To implement the Brute-Force Convex Hull algorithm using Python and identify the boundary points that form the smallest convex polygon enclosing a given set of two-dimensional points.

2. Introduction

A convex hull is the smallest convex polygon containing all given points. It is used in geographical mapping, image processing, robotics, and computer graphics. The brute-force method checks point pairs to identify the edges of the polygon.

3. Problem Statement

Given a set of two-dimensional points, identify the outer boundary points, determine the hull edges, and analyse the algorithm's complexity.

4. Mathematical Principle

For three points $A = (x_1, y_1), B = (x_2, y_2)$, and $P = (x_3, y_3)$ calculate the orientation value:

$$O = (x_2 - x_1)(y_3 - y_1) \\ -(y_2 - y_1)(x_3 - x_1)$$

The sign of O indicates the position of point \(P\) relative to the directed line from \(A\) to \(B\):

- O > 0 : Point lies on one side of the line.
- O < 0 : Point lies on the opposite side.
- O = 0: Points are collinear.

A pair of points can form a convex hull edge when all other points lie on the same side of the line, allowing for collinear points.

5. Algorithm

1. Store the input points as coordinate pairs.
2. Select every possible pair of distinct points.
3. Calculate the orientation of every remaining point relative to the selected pair.
4. Check whether all remaining points lie on the same side of the line.
5. If the condition is satisfied, mark the selected pair as a hull edge.
6. Collect the points belonging to these edges.
7. Remove duplicate points.
8. Arrange the hull points in boundary order.
9. Display the convex hull.

6. Python Program

```python
def orientation(a, b, p):
    return (
        (b[0] - a[0]) * (p[1] - a[1])
        - (b[1] - a[1]) * (p[0] - a[0])
    )
def convex_hull(points):
    n = len(points)
    hull_points = set()
    if n < 3:
        return points.copy()
    for i in range(n):
        for j in range(i + 1, n):
            positive = False
            negative = False
            for k in range(n):
                if k == i or k == j:
                    continue
                value = orientation(
                    points[i], points[j], points[k]
                )
                if value > 0:
                    positive = True
                elif value < 0:
                    negative = True
            if not (positive and negative):
                hull_points.add(points[i])
                hull_points.add(points[j])
    if len(hull_points) < 3:
        return sorted(hull_points)
    center_x = sum(p[0] for p in hull_points) / len(hull_points)
    center_y = sum(p[1] for p in hull_points) / len(hull_points)
    ordered_hull = sorted(
        hull_points,
        key=lambda p: __import__("math").atan2(
            p[1] - center_y, p[0] - center_x
        )
    )
    return ordered_hull
points = [
    (0, 0),
    (0, 4),
    (4, 4),
    (4, 0),
    (2, 2),
    (1, 2)
]
result = convex_hull(points)
```

7. Explanation of the Program

- orientation() determines the relative position of a point.
- Nested loops examine pairs and remaining points.
- Valid edge endpoints are stored without duplicates.
- The centroid and atan2() function help arrange the hull points in boundary order.

8. Sample Output

Input points: [(0, 0), (0, 4), (4, 4),
(4, 0), (2, 2), (1, 2)]

Convex hull points: [(0, 0), (4, 0),
(4, 4), (0, 4)]

The four corner points form the convex hull. The points \((2,2)\) and \((1,2)\) lie inside the polygon and are therefore excluded.

Output

```
Python 3.14.7 (tags/v3.14.7:823f032, Aug  5 2026, 10:51:32) [MSC v.1944 64 bit (AMD64)] on win32
Enter "help" below or click "Help" above for more information.
>>>
========== RESTART: C:/Users/ASUS/Downloads/1.py ==========
Input points: [(0, 0), (0, 4), (4, 4), (4, 0), (2, 2), (1, 2)]
Convex hull points: [(0, 0), (4, 0), (4, 4), (0, 4)]
```

9. Experimental Analysis

The algorithm considers every pair of points and examines the remaining points to check whether the selected line can be a hull edge.

Number of points	Number of point pairs	Approximate orientation checks
5	10	30
10	45	360
20	190	3,420
50	1,225	58,800
100	4,950	485,100

The number of point pairs is calculated using:

$$\frac{n(n-1)}{2}$$

Each pair requires checking up to n-2 other points. Consequently, the algorithm performs approximately cubic work as the input size increases.

10. Time and Space Complexity

Measure	Complexity
Time complexity	O(n^3)
Auxiliary space	O(n) for storing hull points

The algorithm examines O(n^2) pairs and performs up to O(n) orientation checks for each pair. Therefore, its time complexity is O(n^3).

11. Analysis and Discussion

The brute-force method is easy to understand and works well for small point collections. However, its calculations increase rapidly with input size. Graham Scan and Andrew's Monotone Chain algorithms can construct a convex hull in O(nlog n) time. The simple implementation may require additional handling for duplicate or collinear points.

12. Conclusion

The Brute-Force Convex Hull algorithm identifies the outer boundary using orientation calculations. It is suitable for small datasets but becomes inefficient for large inputs because of its O(n^3) time complexity.

➢ Q6 – Exhaustive Search and Combinatorial Growth

1. Aim

To design and implement an Exhaustive Search algorithm for humanitarian airlift planning, identify

the package combination with the maximum humanitarian value without exceeding the payload limit, and analyse the growth in computational effort as the number of packages increases.

2. Introduction

Exhaustive Search is a problem-solving technique that examines every possible candidate solution and selects the best one that satisfies the given conditions. In humanitarian airlift planning, a helicopter has a limited payload capacity, so the selected packages must remain within the weight limit while delivering maximum humanitarian value.

3. Problem Statement

Given n packages and a maximum payload capacity \(W\), select a combination of packages that:

- Does not exceed the payload capacity.
- Maximises the total humanitarian value.
- Identifies the best feasible combination.

The experiment also investigates the growth of candidate combinations and determines a reasonable problem-size limit for deployment.

4. Algorithm Design

- Candidate representation: Each candidate is represented by a binary sequence. A value of 1 indicates that a package is selected, while 0 indicates that it is not selected.
- Candidate generation: Generate every possible subset of the available packages.
- Feasibility check: Calculate the total weight of each subset and check whether it is less than or equal to the payload capacity.
- Evaluation: Calculate the total humanitarian value of each feasible subset.
- Best solution: Retain the feasible subset with the highest total value.

5. Python Program

```
from itertools import combinations
from time import perf_counter
def exhaustive_search(packages, capacity):
    best_value = 0
    best_weight = 0
    best_package_ids = []
    candidates = 0
    feasible_count = 0
    n = len(packages)
    for r in range(n + 1):
        for subset in combinations(packages, r):
            candidates += 1
            total_weight = 0
            total_value = 0
            package_ids = []
            for package in subset:
                package_id, weight, value = package
                total_weight += weight
                total_value += value
                package_ids.append(package_id)
            if total_weight <= capacity:
                feasible_count += 1
                if total_value > best_value:
                    best_value = total_value
                    best_weight = total_weight
                    best_package_ids = package_ids
    return (best_package_ids, best_weight, best_value,
            candidates, feasible_count)
packages = [
    ("P1", 2, 30),
    ("P2", 3, 40),
    ("P3", 4, 50),
    ("P4", 5, 60)
]
capacity = 8
result = exhaustive_search(packages, capacity)
print("Payload capacity:", capacity)
print("Selected packages:", result[0])
print("Total weight:", result[1])
print("Total humanitarian value:", result[2])
print("Candidate combinations:", result[3])
print("Feasible combinations:", result[4])
print("\nGrowth Investigation")
print("Packages | Candidates | Time (seconds)")
for n in [5, 10, 15, 20]:
    test_packages = [
        (f"P{i+1}", 1, i+1) for i in range(n)
```

6. Correctness Verification

Consider the four packages used in the program, with a payload capacity of 8 units.

Package	Weight	Humanitarian value
P1	2	30
P2	3	40
P3	4	50
P4	5	60

The best feasible combinations can be checked independently:

Combination	Total weight	Total value	Feasible?
P1, P2	5	70	Yes
P1, P3	6	80	Yes
P1, P4	7	90	Yes
P2, P3	7	90	Yes
P2, P4	8	100	Yes
P3, P4	9	110	No

The combination P2 and P4 delivers a humanitarian value of 100 without exceeding the capacity. No feasible combination provides a higher value.

Therefore, the expected result is:

Payload capacity: 8

Selected packages: ['P2', 'P4']

Total weight: 8

Total humanitarian value: 100

Candidate combinations: 16

Feasible combinations: 10

This verifies the optimal result for the small test instance. The complete enumeration examines \(2^4=16\) subsets, including the empty subset.

7. Mathematical Prediction of Combinatorial Growth

Each package has two possible states: selected or not selected. Therefore, for \(n\) packages, the number of candidate combinations is:

$$C(n) = 2^n$$

Number of packages n	Predicted combinations 2^n
5	32
10	1,024
15	32,768
20	1,048,576
25	33,554,432
30	1,073,741,824

Every time one package is added, the number of candidate combinations doubles. Increasing the input from 20 to 30 packages increases the number of combinations from just over one million to more than one billion.

8. Experimental Validation

The Python program tests 5, 10, 15, and 20 packages. For each test, it records the total number of candidates and execution time.

The candidate counts are mathematically predictable and independent of the machine:

Packages	Predicted candidates	Expected observation
5	32	Very small search
10	1,024	Still manageable
15	32,768	More processing required
20	1,048,576	Substantially more work

The execution time is measured by perf_counter(). Its exact values depend on the computer and Python environment, so the times printed by the program should be recorded as the actual experimental results rather than assumed in advance.

9. Analysis of Combinatorial Growth

The main limitation of Exhaustive Search is the exponential growth of its solution space. The algorithm must generate and evaluate every subset, even when many subsets are clearly unsuitable.

Adding a single package doubles the number of combinations. Consequently, a relatively small increase in the number of packages can produce a dramatic increase in processing time and computational effort.

10. Deployment Decision

For this experiment, I would set a response-time target of 2 seconds on the computer used for testing. This is a chosen engineering requirement for a prototype, not a universal standard for emergency systems.

The deployment boundary should be established by measuring the program on the target computer and selecting the largest tested input that consistently completes within 2 seconds.

Although the candidate count at 20 packages is 1,048,576, the exact acceptable limit cannot be determined from the mathematical count alone. It must be confirmed using measured execution times. For larger package collections, a redesigned search strategy should be considered.

11. Conclusion

The experiment demonstrates that Exhaustive Search can find the optimal package combination by evaluating every possible subset. The four-package test confirms the correct solution, while the growth investigation shows that the number of candidates increases exponentially as packages are added. Therefore, Exhaustive Search is useful for small instances but becomes impractical for larger humanitarian airlift planning problems.

➢ Q7 – Day-4 Design Challenge: Diagnose, Redesign and Validate

Selected Problem: Q6 – Exhaustive Search for Humanitarian Airlift Planning

A. Diagnose the Computational Bottleneck

1. Original limitation

The original algorithm generates and evaluates every possible package combination. For \(n\) packages, it examines 2^n candidates.

The main computational effort is spent on:

- Generating all package subsets.
- Calculating the weight and value of each subset.
- Checking whether the payload capacity is exceeded.
- Comparing each feasible solution with the best solution found so far.

Many subsets may exceed the payload limit, but the original algorithm still calculates their total weight and value before rejecting them.

2. Evidence from the original algorithm

Packages	Candidates examined
5	32
10	1,024
15	32,768
20	1,048,576

The number of candidates doubles whenever one package is added. This exponential growth is the primary computational bottleneck.

B. Define the Design Objective

The redesign should reduce the number of candidate combinations that require complete evaluation while preserving the optimal humanitarian value.

The measurable objectives are:

1. Examine fewer complete candidate solutions than the original algorithm.
2. Avoid exploring combinations that cannot improve the best known solution.
3. Produce the same optimal value as Exhaustive Search on the same test cases.

4. Record the number of search nodes examined to compare both strategies.

The redesigned algorithm must not sacrifice correctness to achieve faster execution.

C. Design the Revised Computational Strategy

1. Proposed approach: Pruning unnecessary candidates

Instead of evaluating every complete combination, the revised strategy will build a solution one package at a time. It will estimate the maximum value that could still be obtained from each partial solution. If that estimate cannot exceed the best value already found, the remaining combinations in that branch can be skipped.

The design uses the following information:

- Current total weight.
- Current total humanitarian value.
- Remaining packages.
- Best value found so far.
- An upper bound on the value achievable from the remaining packages.

Packages are sorted by value-to-weight ratio to help estimate the upper bound. A fractional estimate may be used for the bound, but actual selected packages must remain whole.

2. Revised algorithm

1. Sort packages by decreasing value-to-weight ratio.
2. Initialise the best solution with value zero.
3. Consider including or excluding the next package.
4. Reject a branch immediately if its weight exceeds the capacity.
5. Calculate an upper bound on the value obtainable from the remaining capacity.
6. If this bound cannot beat the best known value, stop exploring that branch.
7. Otherwise, continue exploring its possible selections.
8. Return the best feasible package combination.

3. Pseudocode

```
BranchAndBound(packages, capacity):
  Sort packages by decreasing value/weight ratio
  best_value = 0
  best_solution = empty set
  Explore(index, current_weight, current_value, selected):
    Update best solution if current_value is better
```

If all packages have been considered:

Return

Calculate upper bound for this branch

If upper bound <= best_value:

Return

If current_weight + next_package.weight <= capacity:

Explore including the next package

Explore excluding the next package

Explore(0, 0, 0, empty set)

Return best_solution

4. Expected trade-offs

The revised strategy introduces extra work for sorting and calculating upper bounds. It may also require additional memory to store the current selection and best solution.

However, it can avoid evaluating large groups of unpromising combinations. Its worst-case time complexity remains exponential, so it does not guarantee fast performance for every possible input.

D. Predict Before Implementing

Before testing the redesigned algorithm, I predict that branch pruning will reduce the number of complete candidate solutions evaluated compared with Exhaustive Search.

The primary performance measure will be the number of search nodes explored. I also expect the redesigned algorithm to return the same maximum humanitarian value for the same input.

E. Implement and Validate the Redesign

The following prototype implements the revised strategy. It counts the search nodes explored and compares the result against the original Exhaustive Search algorithm.

Python Program

```
def branch_and_bound(packages, capacity):
    items = sorted(
        packages,
        key=lambda p: p[2] / p[1],
        reverse=True
    )
    best_value = 0
    best_weight = 0
    best_ids = []
    nodes = 0
    def upper_bound(index, weight, value):
        bound = value
        remaining = capacity - weight
        for i in range(index, len(items)):
            package_id, item_weight, item_value = items[i]

            if item_weight <= remaining:
                remaining -= item_weight
                bound += item_value
            else:
                bound += item_value * remaining / item_weight
                break
        return bound
    def explore(index, weight, value, selected):
        nonlocal best_value, best_weight, best_ids, nodes
        nodes += 1
        if weight > capacity:
            return
        if value > best_value:
            best_value = value
            best_weight = weight
            best_ids = selected.copy()
        if index == len(items):
            return
        if upper_bound(index, weight, value) <= best_value:
            return
        package_id, item_weight, item_value = items[index]

        if weight + item_weight <= capacity:
            selected.append(package_id)
            explore(
                index + 1,
                weight + item_weight,
                value + item_value,
                selected
            )
            selected.pop()

        explore(index + 1, weight, value, selected)

    explore(0, 0, 0, [])

    return best_ids, best_weight, best_value, nodes


packages = [
    ("P1", 2, 30),
    ("P2", 3, 40),
    ("P3", 4, 50),
    ("P4", 5, 60)
]

capacity = 8

original = exhaustive_search(packages, capacity)
revised = branch_and_bound(packages, capacity)

print("Original Exhaustive Search")
print("Candidates examined:", original[3])
print("Best value:", original[2])

print("\nRevised Branch and Bound")
print("Search nodes explored:", revised[3])
print("Selected packages:", revised[0])
print("Total weight:", revised[1])
print("Best value:", revised[2])
```

Validation results

Both algorithms should identify the same optimum for the four-package instance.

Measure	Exhaustive Search	Revised strategy
Optimal package combination	P2, P4	P2, P4
Total weight	8	8
Maximum value	100	100
Search effort	16 complete subsets	Counted search nodes

The original algorithm examines all 16 subsets. The revised algorithm counts every recursive search node, including partial selections. These counts represent different types of work, so the node count should be interpreted alongside the result and, where useful, execution time.

For a fair experimental comparison, test both algorithms with the same package lists and payload capacities. Record the actual measurements produced by the programs instead of assuming that every input will benefit equally from pruning.

F. Defend the Final Design

1. Did the redesign achieve its objective?

The revised algorithm preserves the optimal solution by pruning a branch only when its upper bound cannot exceed the best value already found. For the four-package test, both approaches identify a maximum value of 100.The reduction in search effort must be verified using the measured node counts. Pruning can be effective when the upper bound rules out many branches, but it may offer limited improvement on some inputs.

2. Advantages

- Avoids exploring branches that cannot improve the best solution.
- Preserves the optimal result when the upper bound is valid.
- Provides measurable search-node counts.
- Can reduce unnecessary computation compared with complete enumeration.

3. Limitations and trade-offs

- Sorting packages and calculating upper bounds introduce additional overhead.
- Additional memory is needed for recursive calls and selected packages.
- Worst-case time complexity remains exponential.
- Performance depends on the package weights, values, capacity, and quality of the upper bound.

4. When might the original solution be preferable?

Exhaustive Search may remain preferable for very small package sets because it is simpler to implement,

understand, and verify. Branch and Bound becomes more attractive when the number of packages increases and its pruning strategy eliminates substantial portions of the search space.

5. Conclusion

The original Exhaustive Search algorithm guarantees an optimal loading decision by examining all possible package combinations, but its solution space grows exponentially. The redesigned Branch and Bound strategy uses an upper bound to avoid exploring branches that cannot improve the best known solution.

Both algorithms should produce the same optimal value, while their computational effort can differ. The redesign is therefore a justified improvement when experiments demonstrate meaningful pruning, although it does not eliminate the worst-case exponential growth.